FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	*Calvin Josh D. Garcia*
Section:	*AN4*
Professor:	
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. *Explain the concept of rule-based agents.*
2. *Implement a rule-based agent in Python.*
3. *Interact with an environment by perceiving and acting based on rules.*
4. *Observe and analyze the behavior of an agent in a controlled environment.*

Materials Needed

- *Computer with Python 3.x installed*
- *Text editor or IDE (e.g., VSCode, PyCharm)*
- *Optional: Jupyter Notebook for interactive testing*

General Instructions

1. *This* **Technical Assessment (TA4)** *must be completed individually.*
2. *Follow the Task below.*
3. *Include clear comments in your code to describe its logic.*
4. *Each program should display the correct input, process, and output.*
5. *Ensure that your script displays clear and readable outputs.*
6. *Compile all Python codes and corresponding outputs into one PDF file.*
7. *Save it as* **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. *Submit the* **python file** *and* **PDF** *file.*

Laboratory Task

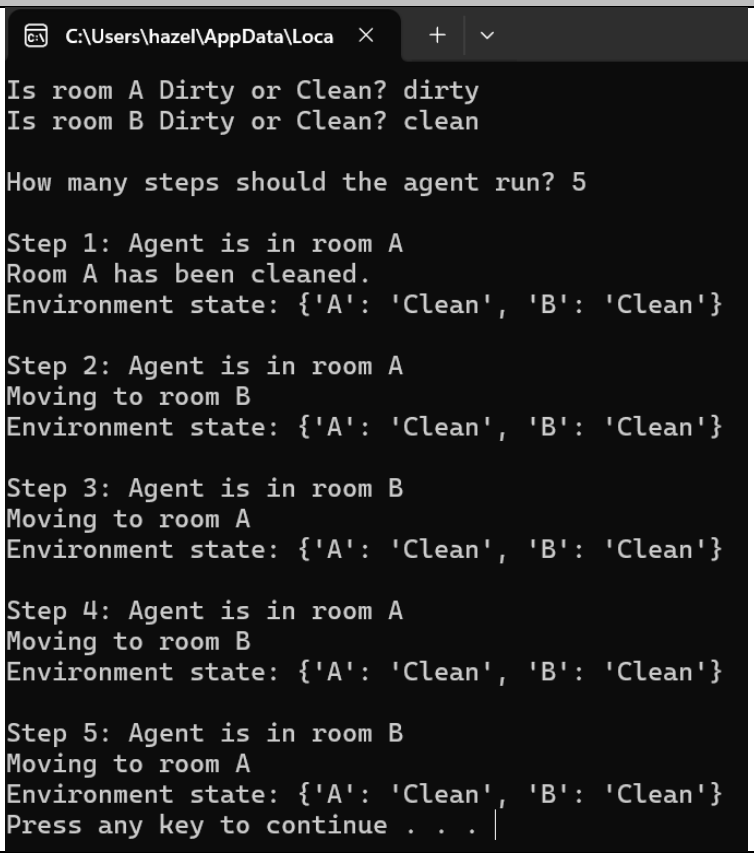
Pre-Lab Preparation

1. *Review the Python concepts:*
   - *Classes and objects*
   - *If-Else statements*
   - *Loops (for loop)*
   - *Functions / methods*
2. *Understand the environment:*
   - *Two rooms: A and B*
   - *Each room can be Dirty or Clean*

Task 1: Set Up the Environment

1. *Initialize two rooms using a Python dictionary.*
2. *Allow the user to set the* **initial state** *of each room (dirty or clean).*
3. *Write a function to* **check if a room is dirty** *and a function to* **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. *Create a class for the agent.*
2. *Implement the* **perceive-and-act method***:*
    - *Rule 1: If the current room is dirty, clean it.*
    - *Rule 2: If the current room is clean, move to the other room.*
3. *Implement a* **move method** *to switch rooms.*

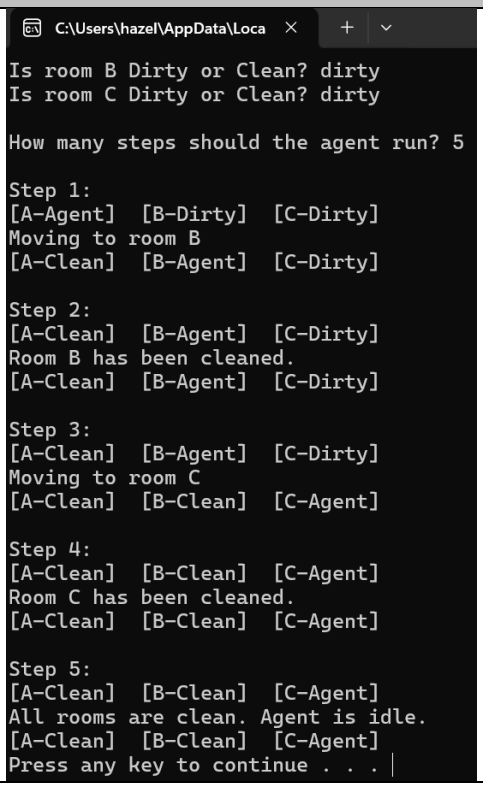
Task 3: Run the Simulation

1. *Ask the user how many steps the simulation should run.*
2. *Run the agent in a loop for the specified steps.*
3. *After each step, print:*
    - *The agent's current location*
    - *The action taken (cleaned/moved)*
    - *Current state of the environment*

Optional (Bonus Task)

- *Add a* **third room (C)** *and update the agent logic.*
- *Make the agent* **randomly choose which dirty room to clean next**.
- *Visualize the agent moving between rooms using a* **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
def initialize_environment(): """Initializes three rooms (A, B, and C) and allows the user to set their initial states.""" environment = {} for room in ['A', 'B', 'C']: while True: state = input(f"Is Room {room} Dirty or Clean? ").strip().lower() if state in ['dirty', 'clean']: environment[room] = state break print("Invalid input. Please enter 'dirty' or 'clean'.") return environment

def is_dirty(environment, room): return environment.get(room) == 'dirty'

def clean_room(environment, room): environment[room] = 'clean'

class VacuumAgent: def init(self, start_location='A'): self.location = start_location

def perceive_and_act(self, environment):
    current_room = self.location
    action = ""
```

```
    if is_dirty(environment, current_room):
        clean_room(environment, current_room)
        action = f"Room {current_room} has been cleaned"
    else:
        self.move()
        action = f"Moving to room {self.location}"

    return current_room, action

def move(self):
    if self.location == 'A':
        self.location = 'B'
    elif self.location == 'B':
        self.location = 'C'
    else:
        self.location = 'A'


def run_simulation(): env = initialize_environment()

start_loc = 'A'

agent = VacuumAgent(start_location=start_loc)

try:
    steps = int(input("How many steps should the agent run? "))
except ValueError:
    steps = 5
    print("Invalid number of steps entered. Defaulting to 5 steps.")


for step in range(1, steps + 1):
    loc, action_desc = agent.perceive_and_act(env)
    print(f"Step {step}:")
    print(f"  - Agent is in room {loc}")
    print(f"  - {action_desc}")
    print(f"  - Environment State: {env}")
    print("-" * 40)


if name == "main": run_simulation()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. *What rules does the agent follow?*
2. *How does the agent decide when to clean and when to move?*
3. *What happens if both rooms are already clean at the start?*
4. *How would you modify the agent to handle* **more than two rooms***?*

Answer the Questions ⇩

1. **Rule 1: If the current room the agent is located in is dirty, it cleans the room.**
   Rule 2: If the current room is already clean, the agent switches/moves to the other room

2. **The agent perceives its current environment state by checking the dictionary value associated with its current room location. If the state evaluates to 'dirty', it executes the cleaning action. Otherwise, if the state is 'clean', it bypasses cleaning and executes the move() method to alternate rooms.**

3. **If both rooms are initially clean, the agent perceives its current room as clean, triggers Rule 2, and continuously alternates (bounces) back and forth between Room A and Room B at every step without changing any environmental states.**

4. **To handle more rooms (e.g., Rooms A, B, and C), the environment dictionary would be expanded to include all room states. The agent's move() method would be updated from a simple binary toggle to a sequential or graph-based traversal strategy (such as checking adjacent rooms or choosing randomly among uncleaned/dirty rooms) to navigate through the multi-room layout effectively.**